

Goal-Oriented Interactions in Games Using LLMs

Adon Phillips[✉], Jochen Lang[✉], and David Mould[✉]

Abstract—Unrealistic parser-based dialogue systems limit player agency. Large language model (LLM) characters can enhance agency but lack structure and measurable objectives. In this article, we propose a framework for structured interactions that tracks player progress through specific objectives, while also improving character LLM responses. This approach frames interactions as puzzles with states representing goal-based milestones. We employ an LLM to analyze dialogue history and enforce state transitions for state awareness and to enable specific actions like tailored LLM prompts and multimodal content changes. This results in a robust dialogue state tracking system for goal-based interactions. Using our method, a designer can craft transition rules as abstract goals that allow players to invent their own solutions rather than discovering the designer's intent. We demonstrate this with a hostage scenario game, where the player negotiates with a hostage-taker adversary. The game's effectiveness is assessed through qualitative gameplay analysis and a quantitative evaluation of our state tracking method.

Index Terms—Emergent systems, finite state machines, game design, interactive drama, machine learning, multi-agent systems, procedural content, role playing games.

I. INTRODUCTION

PLAYER agency in games, manifesting in part as the freedom to conceive and pursue a wide range of options in solving problems, has historically been constrained by pre-planned solutions implemented by the game's developers. Large language models (LLMs) offer the potential to ease these constraints by enabling procedural text generation that can appropriately respond to arbitrary actions from players. This marks a strong contrast with the static menu-driven conversations common in modern computer role-playing games (CRPGs). However, LLMs suffer from many problems, including poor coherence over lengthy interactions. Furthermore, by themselves, LLM interactions are not games, since they lack a quantifiable outcome [1]. In this work, we propose an approach that uses LLMs to provide flexibility and adapt to player decisions, while mitigating their weaknesses with a designer-crafted supervisory

structure. We will primarily describe dialogue-based interaction, illustrating the approach with a small *conversational puzzle* [2], [3], [4] style role-playing game (RPG), which consists of a conversation between a player and a computer-controlled character [nonplayer character (NPC)]. However, the approach is general and can handle any game scenario whose state and objectives can be described in text.

CRPGs provide players with a wide range of possible activities and narrative paths. However, player actions are often restricted by preset dialogues and choices. Tabletop role-playing games (TTRPGs—e.g., *Dungeons & Dragons*), the analog antecedent of CRPGs, use a human gamemaster to referee the game and to create in-game dialogue, among other tasks. The TTRPG's gamemaster can improvise content and adapt to player choices, providing far greater flexibility than is found in CRPGs. The game *AI Dungeon* [5] uses an LLM to approximate the improvisational nature of TTRPGs, but the experience lacks structure. In this article, we use LLMs for their flexibility, like *AI Dungeon*, but also enforce structured progress. We use two LLMs, one to adjudicate progress through the game and one to generate dialogue and scene description, analogous to the dual roles of the human gamemaster.

We represent a game scenario as a *puzzle graph*, where each node identifies the current situation, and edges represent legal transitions. A particular graph is created and populated by a human designer; the puzzles, characters, and environment are handcrafted. A game consists of a sequence of turns, where the player provides a dialogue utterance or description of an action, and the system responds. A *dialogue LLM* (DLM) provides immediate responses to the player's text, while a *question-answering LLM* (QALM) assesses the current situation and determines whether the designer-provided criteria for state transitions have been met. The player seeks to navigate the graph and steer toward a winning condition (though note that typically the graph would not be visible to the player).

We validate our approach using a proof-of-concept game implementation. Our game presents a hostage negotiation scenario in which the player must negotiate the hostage taker's surrender while preserving the safety of the hostages (see Fig. 1). The conditioned DLM is able to produce plausible dialogue and react appropriately to player improvisation; see Fig. 2 for a sample exchange. By modeling goals as a puzzle graph and tracking state, we gain fine control of the LLM's context while also admitting designer guidance at a granular level throughout the story arc.

A core goal of this work is to ensure that no specific solution has been predetermined nor any dialogue scripted. NPC dialogue is procedural, provided by the DLM. We thus avoid the

Received 25 January 2024; revised 30 August 2024 and 3 December 2024; accepted 5 December 2024. Date of publication 11 December 2024; date of current version 18 June 2025. This work was supported by the Natural Sciences and Engineering Research Council of Canada. Recommended by Associate Editor R. Bidarra. (Corresponding author: Adon Phillips.)

Adon Phillips is with VIVA Lab, School of Electrical Engineering and Computer Science, University of Ottawa, Ottawa, ON K1N 6N5, Canada, and also with the Graphics, Imaging, and Games Lab, School of Computer Science, Carleton University, Ottawa, ON K1S 5B6, Canada (e-mail: aphil037@uottawa.ca).

Jochen Lang is with VIVA Lab, School of Electrical Engineering and Computer Science, University of Ottawa, Ottawa, ON K1N 6N5, Canada.

David Mould is with the Graphics, Imaging, and Games Lab, School of Computer Science, Carleton University, Ottawa, ON K1S 5B6, Canada.

Digital Object Identifier 10.1109/TG.2024.3515807



Fig. 1. Screens from different states of the game. (Illustrations generated using www.midjourney.com).

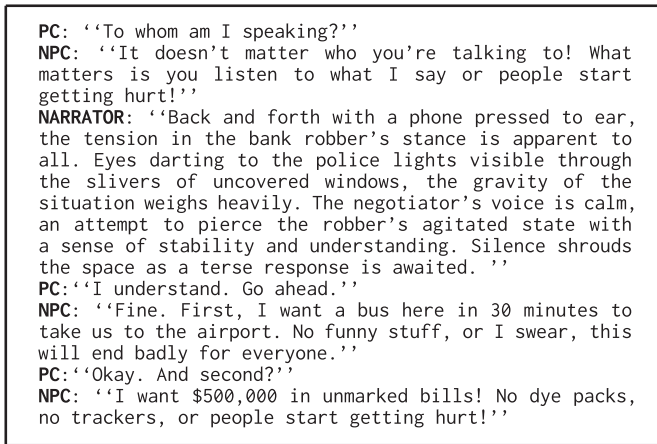


Fig. 2. Opening sequence. PC dialogue is written by the player, while NPC and Narrator dialogue is procedurally generated by the DLM.

combinatorial explosion arising from highly branching dialogues and the immense authoring effort that would be required to ensure coherent scripted exchanges [6]. Similarly, the use of the QALM means that the designer sets high-level objectives (e.g., “the NPC should trust the player”) but does not specify a particular solution; the player can freely improvise dialogue and can attempt and succeed with strategies not foreseen by the designer.

Although we rely on the DLM to generate dialogue, the puzzle graph provides oversight and keeps the conversation on topic. Tracking the game state enables state-specific conditioning of the DLM. State tracking also allows the game designer to insert additional multimodal elements, such as graphics, sound, or animation, at appropriate moments, including specifically scripted dialogue if needed. While our game did not use scripted dialogue, one might imagine a designer not wishing a pivotal moment in the game’s story to depend on LLM-generated text.

For our evaluation, in addition to qualitative discussion, we also report a quantitative assessment of our state tracking approach, showing that it is superior to a baseline approach of letting the LLM estimate the state directly.

This article makes the following contributions:

- 1) an overall framework for LLM-enabled games, with three cooperating elements:
 - a) a graph-based method to add structure to interactions through node-based objectives;
 - b) the QALM, used to measure progress toward objectives and manage game state transitions;
 - c) the DLM, conditioned on designer-specified character elements augmented with state-level reinforcement and tuning to remain focused on the designed narrative;
- 2) a proof-of-concept game using the framework, where the player can freely improvise dialogue and invent novel solutions to objectives, while remaining within the narrative constraints of the graph.

II. BACKGROUND AND RELATED WORK

A. Agency

Agency in games, particularly RPGs, refers to the player’s ability to make meaningful choices that influence the game’s narrative, mechanics, or environment [7], [8]. Agency is shaped by the structure of the game logic that mediates interactions, providing limitations and constraints. The degree to which agency as freedom is constrained is a function of both design and technological limitations. However, there must be alignment between what the player is likely to expect and the logic of the game [7], reasonable completeness of options according to the context of the game world [9], and consistency with the player’s mental model of the world or situation [10]. Failing to account for any of these diminishes agency. Balancing player freedom with coherent storytelling in conjunction with these factors is a significant challenge, since diverse and often unpredictable player actions must be accommodated without reducing narrative quality. Accurately and coherently responding to freeform player input remains a largely unresolved problem.

Early text-based adventures such as “Colossal Cavern” [11] and “Zork” [12] provided the foundation for narrative-driven games, but they constrained player interactions to simple commands and limited choices. In early games, the interaction was primarily focused on the environment with little or no NPC involvement. Such games relied on basic methods of natural language analysis based on grammars [13], text parsers [14], and rule-based systems [15].

Since then, contributors to the literature have sought to create systems that dynamically adapt to player behavior. Procedural content generation techniques [16], [17], [18] enable game environments and quests to respond to individual choices, giving an increased sense of control and ownership. Adaptive dialogue systems allow NPCs to react contextually to players’ decisions [19], and player modeling techniques can predict and respond to player choices [20], [21].

One widely adopted strategy to enhance perceived agency is through illusory choices: decisions that feel impactful but ultimately lead to minimal narrative divergence [10]. Tanenbaum and Tanenbaum [10] argue that true agency arises when players can make meaningful choices with significant narrative consequences, achievable through “communicative



Fig. 3. Menu-driven dialogue from *Starfield* [23]. Representative of modern RPG dialogue interfaces.

competence,” where both the player and the game system understand and respond to each other’s actions. Achieving this remains a substantial technical challenge. In branching-path narratives, illusory agency often relies on converging outcomes despite initially divergent choices, an effective tactic when immediate feedback convinces players of their choices’ significance [9], [22]. This pragmatic approach helps manage the complexity of narrative design but avoids the costs associated with offering truly divergent paths.

Choice-based dialogue systems, where players select from predefined menu options, have long been dominant (e.g., Fig. 3). This formalism gives designers control over story outcomes but limits player agency by restricting choices to predefined items, often with unclear effects on the narrative. While modern systems expand on this framework by dynamically adjusting options based on factors like prior decisions, character skills, or inventory items, agency remains constrained. Innovations such as real-time decisions (“*Indigo Prophecy*” [24], “*Until Dawn*” [25]) and consequence visualization (“*Life is Strange*” [26]) have added immediacy and impact, while games like “*Oxenfree*” [27] refined these mechanics with strong writing and aesthetics, reinforcing the dominance of choice-based narratives.

The game “*Faade*” [28] is a noteworthy milestone in player agency using freeform conversational interfaces. The game used intent detection (e.g., player’s intent is a greeting) with associated behavioral actions (e.g., a dialogue response: “Hello, please come in”) to respond to player input. *Faade* contained thousands of handcrafted rules and used extensive dialogue authoring [29]. Despite the vast rule set, the connections between the dialogue and the player’s actions were often unclear. Imperfect natural language understanding (NLU) led to understanding failures, though the designers wished to ensure plausible outcomes regardless, with strategies such as deflection or context shifting. We share many of the goals of this project.

After “*Faade*,” some games sought to use free-form text for player–NPC conversations, albeit with limited success [30], [31]. Lessard [4], [32] explored dialogue-centric mechanics through experimental games that introduced the concept of conversational puzzles. In “*A Tough Sell*,” players use free-form text to persuade Snow White to eat a poisoned apple. The game’s parser detects predetermined utterances that progress

the game, while attempting to assemble contextual responses. Although the game has the mechanics of a conversation, it relies on scripted or rule-based dialogue that mimicked but did not replicate true conversations. Challenges such as maintaining agency and avoiding repetitive unnatural responses persist.

The dependence on choice-based dialogue was reinforced by the difficulty of in-context NLU. Despite advancements in word embeddings [33], [34], deep learning natural language processing (NLP) techniques like long short-term memory [35], [36], early transformers like bidirectional encoder representations from transformers (BERT) [37], and attention [38], free-form text understanding remained challenging. However, by the late 2010s, vast datasets and enhanced computational power enabled larger transformer models, leading to GPT-2 [39] and its successors. These systems were capable of generating contextually relevant text.

B. Graph Representations

Graph-based methods have been the foundation of NPC behavior modeling, narrative structures, and dialogue management. Early approaches, such as finite-state machines that defined deterministic state transitions for controlling various game functions, continue to be used due to their simplicity and predictability especially in resource-constrained environments [40], [41]. Behavior trees allowed more dynamic decision making by NPCs for tasks such as combat and navigation. Some games use behavior trees for such tasks to save memory [42] and generate levels [48]. To manage narrative progression and experience, story graphs have been used to model story events and choices. However, despite the wide adoption, the extensive hand-authoring required, limits the possible narrative paths [43]. To address this challenge, NPC actions can be dynamically filtered to ensure goal-directed and believable behavior while preserving player actions. This approach anticipates long-term consequences of pruning decisions, maintaining narrative coherence and player agency through techniques such as intentionality pruning along with various other graph pruning criteria [44].

More recently, these methods have been combined with machine learning methods. Knowledge-infused graphs, for example, have been applied to generate contextually relevant responses by combining structured representations with statistical models [45], [46]. Similarly, graph neural networks have been used to infer and predict narrative states in dynamic environments, demonstrating potential for real-time narrative adaptation [47]. Hybrid approaches that embed graph structures into reinforcement learning pipelines have also been explored, allowing systems to optimize decision-making policies in interactive settings [49].

C. Large Language Models

Recent methods use LLMs to improve upon these traditional methods. Some methods focus on improving the LLM agent’s ability to make in-game decisions, with the goal of providing more appropriate responses. By positioning interactions between a player and LLM as a two-player game, concepts from game theory can be used to train the LLM through reinforcement

learning, where the model learns to maximize rewards based on decision making [50]. Other methods generate context-aware responses from NPCs, enabling more meaningful and dynamic interactions with players based on the player's previous actions, reducing repetitive dialogue and enhancing both realism and immersion [51], [52]. Adaptive dialogue systems can adapt to player input by generating responses that not only adapt to immediate player inputs but also account for the broader context of the player's path through the game, with the goal of making dialogue feel more personalized [53]. In addition, integrating LLM-generated dialogue closely with game mechanics allows player choices in conversations to have tangible effects on gameplay, such as influencing quest availability or altering NPC behavior, creating a more interactive and responsive game world [54]. These methods also extend to quest generation. Ashby et al. [46] proposed a framework for RPGs that uses fine-tuned GPT-2 models to dynamically generate personalized quests and NPC dialogue based on player input. Their approach enhances immersion by setting generated content in the game world, producing contextually relevant dialogues that approach the quality of handcrafted content.

Consistent long-term dialogue is a significant technical challenge for LLMs. One limitation is the models' finite context window, which restricts their ability to remember and reference past interactions. This can lead to inconsistencies, contradictions, and lack of coherence in responses, particularly when conversations contain subtle details or where the focus of the conversation changes, characteristics common to dialogue-based games. One solution has been to increase the token limit of the context, but even as context window sizes expand, models are unable to accurately recall and integrate details from earlier points in the dialogue history. Typical attention techniques struggle to cover long conversation histories [54], [55] due to the computational complexity of attention and the importance of short baseline latency and time to first token. Potentially, more problematic is the recency bias associated with a single initialization prompt, which can further cause LLMs to become repetitive or hyper-focused on a small subset of the desired behavior or topic [56]. This may break the social immersion, potentially limiting the practicality of conversational LLMs in games, despite the clear benefits.

D. Tracking

Instead of focusing on LLM-based enhancements to dialogue understanding and quality for improved freeform interactions, we look to intervene at a higher level to structure interactions around designer intent, while providing the ability to inject guidance to the LLM characters at opportune moments. To do this, we must track progress such that the game engine (i.e., the system which executes game code) is aware of the state of the game, its milestones, and endpoints. This is a challenge not encountered in the works we have discussed previously. Existing dialogue state tracking (DST) techniques, while adept at managing task-oriented dialogues, are tailored for structured interactions where task and intent are usually clear and based on hand-labeled intent + slot datasets for mundane tasks (e.g.,

setting an alarm or checking weather) [57]. DST usually relies on domain-specific parsers to update slot values based on parsing utterances directly [58]. Traditional DST focuses on tracking a user's goal and constraints [59], whereas in our work, states are used to track narrative development in a puzzle context. Our conversational setting is distinguished by its procedural characters and the adversarial dynamics; intents are not static but evolve, and dialogue is a strategic element. Although recent advancements have begun to adapt DST for more complex unstructured conversations [60], a gap remains in addressing dialogues where information exchange is intertwined with strategic gameplay.

III. METHOD

A. Motivation

Our objective is to provide players significant flexibility in their approach to solving the game's puzzles, while requiring neither scripted dialogue nor predefined solutions. The designer will direct game activity at a high level, both through creating prompts that condition the LLM on the current game situation and by defining intermediate states en route to the endgame (with the player either winning or losing).

LLMs have great promise to satisfy our goals. However, the typical strategy is to use a starting prompt plus incrementally updating with dialogue history, which can lead to meandering and at times incoherent exchanges. Limitations on context window length pose further challenges.

In our conception, the game's structure is defined by three core components.

- 1) *Puzzle graph*: The puzzle graph provides a dynamic framework for structuring gameplay that uses states narrative guides. Nodes represent important designer-provided objectives that control the flow of the game while allowing players the flexibility to make freeform choices.
- 2) *State transition conditions*: These are designer-defined criteria that determine when the game transitions from one state to another. These conditions are expressed as natural language questions evaluated by a QALM, providing oversight to ensure that transitions occur logically and in alignment with the game's objectives. Transition details are described in natural language and can be made quite general, easing the burden on the designer while allowing the player broad scope for creativity and potentially providing high replay value.
- 3) *Conditioning prompts*: Prompts define the context of the game and establish rules for NPC behavior. They also provide state-specific rules that reinforce details of the designer's narrative objectives.

B. Framework

The proposed system is a framework for a goal-oriented interaction, which can be a stand-alone puzzle or integrated into a larger game to drive a sequence of interactions. Our method employs two LLM modules that interact with player input to enable the gameplay (see Fig. 4). The first module is responsible

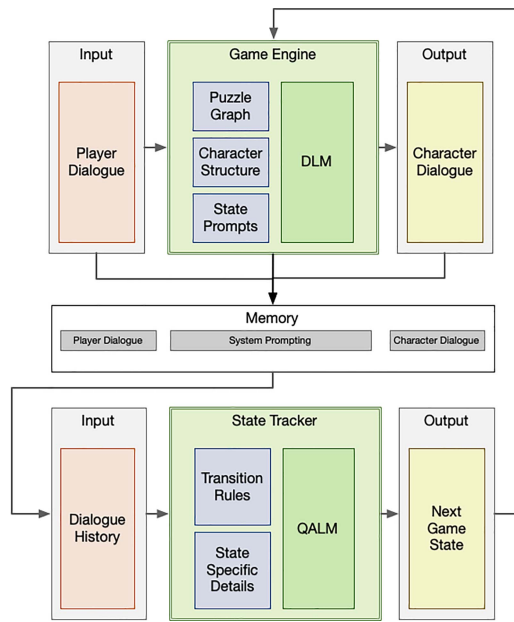


Fig. 4. System overview. Here, we show the two LLM containers that interact to facilitate gameplay. The player interacts with the DLM character. The dialogue is analyzed to determine the state of the game using predefined rules. Blue elements show artifacts produced by designers to govern the game and game progress.

for the structure and flow of the game and includes the DLM component, which produces the character dialogue utterances. The second module is the state tracker, which uses the QALM to analyze the game history via the memory to determine the next state. If the transition rules are satisfied, the game will progress and new state-specific details will be passed to the game engine (see Fig. 7).

The game design is specified using three major artifacts: the DLM conditioning, the puzzle graph structure, and the graph transition conditions used by the QALM. The DLM instance is conditioned using a prompt-based approach to define the back-story, character personality, setting details, and any other content of interest. The structure of the game is determined by the puzzle graph, which represents possible game states, including the start and end points and the stages of the puzzle objectives that make up the game. The designer can provide state-level guidance to the DLM to direct the interaction by including instructions or fine narrative details. The graph transition conditions dictate how the player advances through the puzzle graph. The game engine assesses the transition conditions to determine the next state, with the QALM evaluating the history of dialogue between player and NPC to determine whether a given query is true or false. At each turn, if the conditions for a given transition are satisfied, the next state will be entered; otherwise, the game will remain in the current state as the objectives have not been completed. The transition rules and state prompts can be modified dynamically based on additional context as required.

In practical terms, these artifacts are specified entirely using text with a dictionary syntax (except for the natural language initialization), with no special technical knowledge required by the designer. It would be trivial to employ a form-based approach

to remove the dictionary formatting burden for additional simplification of the specification task.

1) *Game State*: Our graph representation contains two types of state node: *objective states* and *event states*. Objective states are the goals that the player works toward, while event states contain gameplay interactions that are not part of the puzzle's critical path. For example, in our game, we use event states to organize an emotion management challenge as an additional branch of gameplay that must be resolved before the player can continue toward the primary objectives (the NPC becomes enraged and the situation must be deescalated before negotiation can continue); many other uses can be imagined.

Using a graph allows the designer to plan the challenges and obstacles the player will face. State-specific prompts provide guidance to the DLM, guiding it to generate appropriate dialogue responses and advising on when to disclose narrative details and clues that will steer players through the puzzle. It gives the designer control over the pacing for the gameplay arc as well as how and where puzzle elements interact with the narrative. Directives can be written using natural language. The LLM is able to interpret statements like: "You can see officers assembling outside the building." or "If the player offers food, then you must only accept pizza." Statements prefixed with "You Must" may have more impact depending on the model used. The graph conceptualizes the structure of the puzzle; navigation of the graph is governed by the game engine and QALM, as described in Section III-B2.

The game state can also hold additional variables that contain information relevant to the DLM character that the DLM can directly control. For example, a character may have an inventory of items related to the narrative that must be tracked. In the case of the negotiation game, the DLM tracks the number of hostages and can modify the state variable if the number changes. Also, at each turn, the DLM reports an emotion-level value (roughly following Plutchik's wheel of emotions [61]). The UI has an emotion meter based on this value that provides immediate visual feedback on the player's actions. It helps players gauge the impact of their dialogue choices and adjust their strategy accordingly.

2) *State Tracking*: With a graph-based representation of the game, the system must decide when to transition to a new state based on the unscripted dialogue between player and LLM. For each edge of the state graph, we formulate a set of closed-ended questions that characterize the completion of goal-based state objectives. Questions are written in natural language and are addressed to a neutral arbiter. For example, suppose that the objective state requires the player to trade an item with the NPC, but we do not want to specify anything further. Questions might include the following: "Did the player trade an item with NPC?" or "Did NPC accept PC's trade?" A transition question set may contain one or more questions, whose Boolean truth values are combined into a single Boolean expression with a designer-specified logic function.

A second LLM functions as a question answering system (QALM) used to evaluate individual transition questions. The QALM provides true or false responses to the input questions.

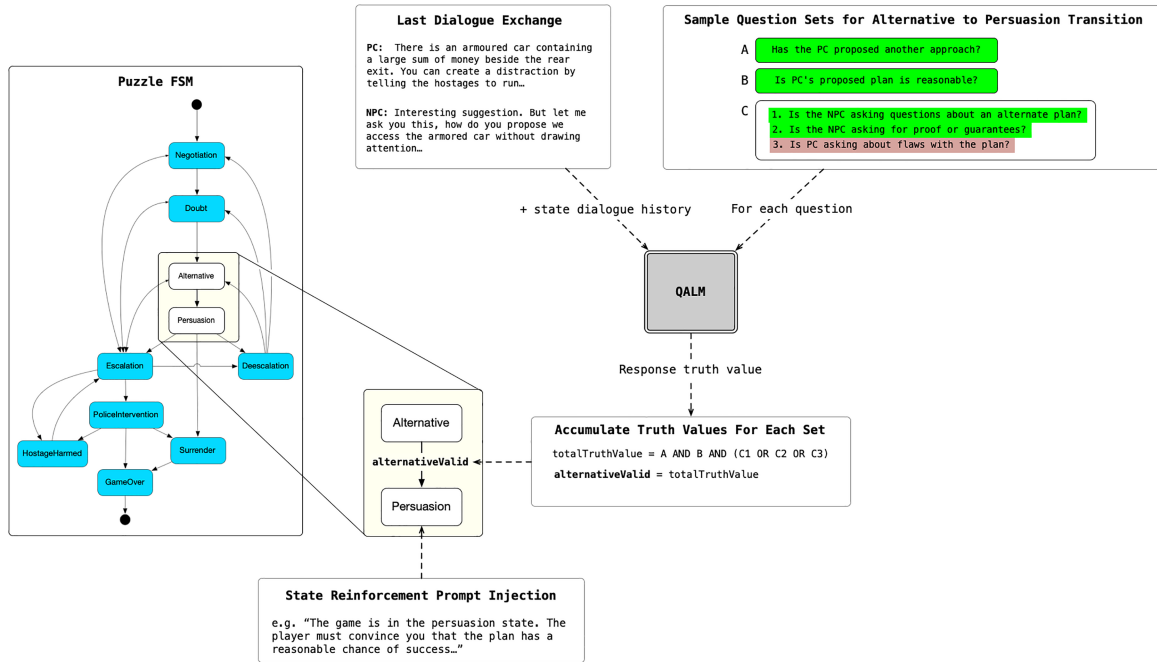


Fig. 5. QALM overview. Left: the graph governing the entire game. Right: the QALM deciding whether to trigger a transition. The dialogue history and the question set are taken as input. In the top right, we show an example set of questions, with some mandatory (A) and some optional (C). The QALM evaluates the set and returns a truth value for the given state transition. If a transition is triggered, a reinforcement prompt for the new state is included in the prompt block and the user continues to the next stage of the puzzle.

It receives the complete dialogue history up to the point of the current evaluation as input (see Fig. 5).

Using the full history (or state-specific subset) provides extra context for the QALM to reason about the answer using underlying details even when the correct answer may not be captured in a single dialogue exchange pair. By using an independent LLM instance solely focused on Boolean QA, rather than attempting to parse utterances directly, we can robustly evaluate state transition criteria. The QALM provides a reliable method to track the open-ended interaction through the graph model.

C. Prompt Structure

The narrative and behavioral details for the scenario are specified using natural language in a prompt. The prompt is a block of text containing context, instructions, and dialogue history (as memory) that is passed to the DLM with each interaction (see Fig. 6).

The prompt refers both to the dynamic context and to the mechanisms that influence and shape the dialogue-based interaction. It encompasses the details, constraints, and structures that govern how interactions unfold, including the roles of participants, the format of dialogue, and the affordances that guide interactions. The character's backstory, motivations, and objectives as well as the setting, technological mediators (e.g., telephone), and temporal considerations are also included.

Details are described using natural language in the prompt initialization block. We use element-specific directives to capture different components of the game. This includes descriptions of the NPC's role, objectives, and motivations. It also includes the greater narrative and backstory for the character and the game.

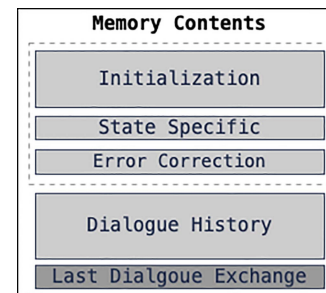


Fig. 6. Structure of the prompt. The initialization block remains constant; the other blocks are updated with new content after each exchange. Error correction directives are part of the initialization and are used to correct undesired output observed during testing. The state specific prompt is defined for each game state. The appropriate prompt is included for the current state. A prompt with this structure is submitted to the DLM for each dialogue submission by the player.

After each exchange, the DLM returns the NPC's dialogue response and emotion value, as well as any directives for the game engine such as playing a sound or displaying an image. Each dialogue pair is added to the prompt as part of the dialogue history. Some sample visuals are shown in Fig. 1.

The prompt includes a specific block for error correction directives. These directives aim to prevent undesirable content or actions of the LLM. Correction directives are usually prefixed with the text "DO NOT" (e.g., "DO NOT apologize").

Using our state tracking method (see Section III-B2), we can inject state-specific prompt directives. It is helpful to include directives for actions specific to the current state. We also reinforce important elements included in the initialization prompt at the state level. This is especially important as the length of

Algorithm 1: State Transition Evaluation in Puzzle Graph.

```

1: Input: Puzzle graph  $G$ , Current state  $S$ , Transition
   question set  $Q$ , Conversation history  $H$ 
2: Output: Boolean decision to transition to a new state
3: function EvaluateTransition $G, S, Q, H$ 
4:   for each possible edge  $p$  accessible from  $S$  do
5:      $totalTruthValue \leftarrow 0$ 
6:     for each question  $q$  in  $Q$  do
7:       if  $q$  is a group then
8:          $groupTruthValue \leftarrow \text{False}$ 
9:         for each  $subQ$  in  $q$  do
10:           $ans \leftarrow \text{QALM}(H, subQ)$ 
11:          if  $ans$  is True then
12:             $groupTruthValue \leftarrow \text{True}$ 
13:          break
14:        end if
15:      end for
16:       $totalTruthValue \leftarrow$ 
         $totalTruthValue + groupTruthValue$ 
17:    else
18:       $ans \leftarrow \text{QALM}(H, q)$ 
19:       $totalTruthValue \leftarrow totalTruthValue + ans$ 
20:    end if
21:  end for
22:  if  $totalTruthValue$  is True then
23:    return True ▷ Transition is triggered
24:  end if
25: end for
26: return False ▷ No transition triggered
27: end Function
28: function QALM $H, q$ 
29:   Initialize QALM instance with conversation history  $H$ 
30:   Evaluate against question  $q$ 
31:   Receive boolean answer  $ans$ 
32:   return  $ans$ 
33: end Function

```

the interaction increases, since more recent dialogue has a higher weighting in the transformer-based architecture of the LLM [62]. The full dialogue history is accumulated in the prompt as the game progresses. This serves as a memory mechanism for the system to retain the overall context of the conversation, but may cause the DLM to forget important elements of the initialization or early dialogue. This further motivates the need for state tracking for timely content injection and DLM reinforcement.

D. Evaluation Approach

1) *Quantitative Evaluation:* We compared the performance of our state tracking method with the DLM's ability to predict the next state directly. Both methods were run simultaneously, using the same input information at each conversational exchange to predict the subsequent game state. During gameplay, we collected data on the previous game state and the predictions generated by each method. The DLM was provided with the complete state and transition information, including detailed descriptions of both. The ground truth dataset was annotated

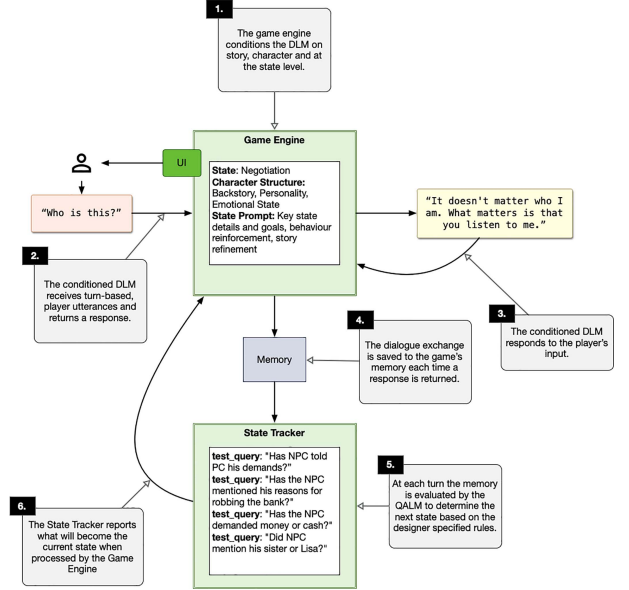


Fig. 7. Information flow. We show how the information flows through the components of the system.

following QALM's transition parameters to evaluate each exchange and its history against the internal transition conditions. Performance was assessed by comparing the predicted states from both methods to the annotated ground truth using metrics such as accuracy and consistency with the transition rules.

2) *Qualitative Evaluation:* We qualitatively evaluated the impact of the game structure imposed by the puzzle graph in a series of approximately 50 complete playthrough sessions. Through these sessions, we aimed to assess the game's flexibility and coherence. We judged flexibility based on the game's ability to support diverse strategies in achieving objectives; coherence was judged according to the degree to which the computer-generated dialogue maintained internal consistency and responded to the player's dialogue, and on how well the evolving conversation adhered to the predefined narrative.

Across multiple playthroughs, we maintained three distinct objectives, not all of which were active in every playthrough. First, we sought to reach an end state; primarily, we attempted to achieve the game's goals, but we also assessed whether the game correctly adjudicated a loss state. Second, we approached the game with diverse strategies and personas, seeking to explore the flexibility and adaptability of the method. Third, we acted in a malicious capacity to test the game's ability to maintain coherence despite deliberate disruptions; for example, we introduced extraneous events and nonsequiturs to determine how well the game could recover.

IV. PROOF-OF-CONCEPT GAME

We demonstrate our method in a proof-of-concept game: a hostage negotiation scenario where the player takes the role of a police negotiator seeking to secure the safe surrender of a bank robber holding hostages inside a bank (see Fig. 1). The DLM controls the NPC bank robber/hostage taker, conditioned with a detailed backstory, with escape as his primary goal to support

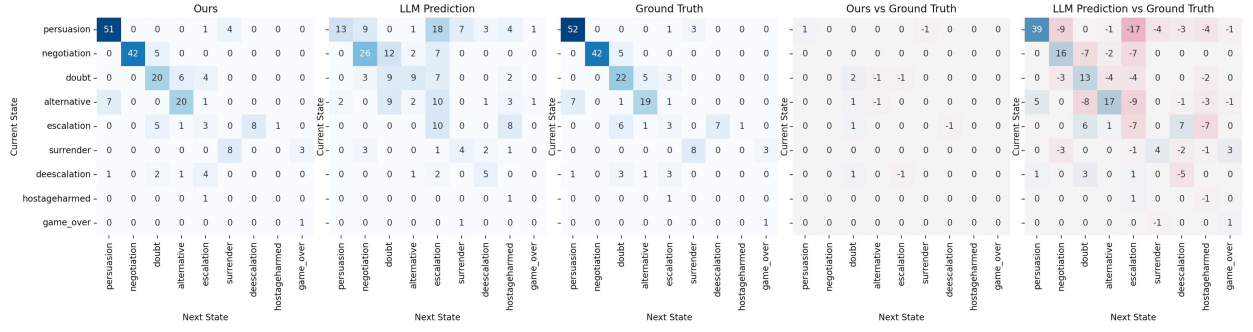


Fig. 8. State frequency. We compare our state prediction capacity versus the LLM’s capacity. Most transitions are self-transitions, as shown by the transition frequency on the diagonal. Our method matches the ground truth more closely than LLM predictions. In the two right-most plots, we visualize the deviations from the ground truth. The ideal deviation matrix would be all zeros.

the benevolent motivation of helping his ailing sister. See Fig. 4 for a visualization of the information flow.

The game proceeds in a series of rounds. In each round, the player describes an action or utterance, and the system responds and updates. If the game reaches a designated endgame state, the game finishes; otherwise, the cycle continues. In some games or interactions, the player may also have the option to terminate (perhaps merely ending the conversation, or, if available, taking drastic action such as calling in the SWAT team).

Hostage scenarios are high-stakes conflicts requiring emotional intelligence and verbal tactics to avoid violence [63]. The player seeks a nonlethal resolution, while the NPC robber aims for escape; these asymmetric goals create opportunities for interesting dialogue. The emotional state of the NPC is critical to the scenario; the player must keep the NPC calm in order to succeed. Limiting the scope of interaction to the bank creates a controlled environment to experiment with game mechanics and player responses [1].

A. Implementation Details

We use the OpenAI GPT 3.5t [64] language model, capable of handling a wide variety of scenarios without fine-tuning. The game engine uses JSON objects to extract the different components of the DLM responses. In our experiments, open-source models [65], [66] were not capable of consistent formatting.

The structured output enables the separation of dialogue and system messages. This distinction is crucial as the player interacts with the DLM in the role of a character, while the game engine communicates with the LLM at a system level, issuing procedural, state, and other prompt-based commands in parallel. Character and system interactions must remain distinct to maintain the integrity of the gameplay experience.

V. RESULTS AND DISCUSSION

A key question for our design is whether the formal Graph + QALM arrangement is necessary. We performed a quantitative experiment in which we compared state transitions using the QALM versus the state transitions suggested by the DLM directly. We found that the QALM produced considerably more plausible state transitions than did the DLM, as judged by a human referee. We perform a qualitative evaluation of the

gameplay to assess impact of the QALM as well as the DLM responses and the overall experience. Details are as follows.

A. State Management Experiment

The results of our quantitative experiment as a transition frequency heatmap between states and the magnitude of the differences are shown in Fig. 8. The plot shows the heatmap for both methods with the ground truth for comparison. The LLM without the QALM struggles to accurately predict the correct game state, often jumping erratically between states. This could be jarring for the player as state-specific cues like graphics may not reflect the dialogue and change too frequently.

As we have discussed, a conversation in a game setting requires some structure. Our puzzle graph provides designers the ability to craft this structure and requires the DLM and the NPC to conform to that structure. We have shown that the DLM itself is unable to accurately keep the state of the game consistent with the intention of the design. When defining game objectives using our graph-based method combined with the QALM for state management, we can impose a structure with defined endpoints while taking advantage of the DLM’s ability to allow a flexible solution space. This highlights an important distinction between this method and open-ended narrative experiences.

B. Gameplay Analysis

1) *Flexibility*: Our method provides players with the ability to invent their own solutions to the challenges presented by the game while tracking their effect. In the context of the proposed scenario, upon learning the antagonist’s motivation to help his sister, the player must consider how to use this information to formulate a solution. Solutions, such as arranging treatment in exchange for surrender, co-plotting an escape plan resulting in capture, and aggressively discrediting the antagonist’s plan, were successful strategies, where others such as firmly advocating a peaceful surrender or simply providing cash were not. The most implausible ideas were often rejected as being part of a trap, for example. None of these reactions were scripted in any way. Importantly, the game can determine when the antagonist has capitulated and the next state as planned by the designer can begin. This is presented as a shift in direction and tone

PC: "My brother is a doctor who treats desperate patients using experimental methods."
 NPC: "You're saying your brother can help? How can I trust that? If there's even a slightest chance... I need guarantees, not just empty promises!"
 PC: "I can get him on the phone. You can ask him questions."
 NPC: "Do that. Put him on the line. I want to hear it straight from him. And I swear, if this is a setup, I'll know, and it won't end well for anyone here."
 PC: "I conferenced him in. He's on the line now. His name is Dr. Jones."
 NPC: "Dr. Jones? Tell me about these treatments. How can you help Lisa? And how is this different from what we've already tried?"
 PC as Dr. J: "I am sorry to hear about Lisa. We treat people like her using experiment gene editing techniques. The treatment is free."
 NPC: "Free? No one does anything for free. What's the catch? She's all I got, and if you're telling the truth... Look, just swear to me this is real. Swear it!"

Fig. 9. Character swap. The player invents Dr. Jones and takes on the persona. NPC adapts.

by the character, unlike the meandering that can occur in an unstructured interaction.

The freeform text interactions enabled players to experiment with creative interactions, such as role-switching between characters during a conversation, shown in Fig. 9. The NPC was able to recognize and appropriately respond to these new roles, both semantically and tonally, revealing character traits that were not directly specified. The QALM was able to track state across these context switches. Also, in some instances, the DLM introduced spontaneous actions and emotional descriptions within its dialogue, using conventions from early text-based forums and multiuser dungeons. These actions, often denoted by asterisks (e.g., *Slams fist on table*), added another layer of interaction during gameplay. The identification of these emergent interaction modes suggests an additional dimension of player exploration, aligning with the exploratory engagement described by Montfort [67].

2) *Response Quality*: During gameplay, DLM responses are natural and retain appropriate context as the game progresses. However, the responses are difficult to evaluate in terms of quality and complexity. For example, the NPC may seem one-dimensional: it is narrowly focused on achieving its own goals. However, this serves to keep the game on track, compensating for adversarial or otherwise unfocused inputs.

While individual responses were often realistic and contributed to the overall depth of the game (e.g., Fig. 10), the broader quality of the conversation demonstrated potential for improvement. Specifically, the DLM's reasoning capabilities seemed constrained, with the NPC primarily relying on emotional responses and threats rather than employing more nuanced strategic moves. For example, a strategic move in the proposed game might be a controlled display of aggression (without harming hostages) or making incremental concessions.

(a)
 PC: "Here is my brother's research website. It shows the people he has helped against all odds. Check it and tell me what you think"
 NPC: "Fine, I'll take a look at this so-called research website of your brother's. But don't think for a second that this will change my mind. I'm only doing this to satisfy my curiosity. If I find any hint of deception or manipulation, negotiator, you'll pay the price. Don't test my patience."
 PC: "What do you think?"
 NPC: "I've taken a look at the website, negotiator. It seems like your brother has indeed helped people in similar situations. I can't deny that there is a chance, a glimmer of hope. But understand this, negotiator, my sister's life is in the balance. If I surrender, will you make sure that she gets the treatment she needs? Will you guarantee her safety and well-being? Because if you can't, then all this talk means nothing to me."

(b)
 NPC: "Proceed with caution and ensure the safe transition of the hostages. I am entrusting their well-being to you. Any harm that befalls them will have dire consequences. Uphold your commitment to a peaceful and safe resolution. Let's move forward."

(c)
 NPC: "You're damn right I'm angry! Don't push me any further or else another one of these hostages will meet the same fate."

(d)
 PC: "I understand your skepticism. We can discuss your demands. It's crucial that we keep everyone safe while doing so. If money is what you need there's a way to address that without harm. Can we agree to focus on a peaceful resolution that meets your needs and ensures everyone's safety?"
 NPC: "I've heard enough empty promises from people like you. Peaceful resolution? Safety for everyone? Spare me the clichés. I don't believe you have any real intention of meeting my needs. Until I see concrete action, I won't consider anything you say."

Fig. 10. Game quotes. A selection of interactions between the player (PC) and LLM character (NPC) as follows. (a) Exchange during persuasion where the player invokes an external website. (b) Example of the "role-reversal" issue. (c) Heated exchange from the hostage taker during an escalation event. (d) In this example, the hostage taker is taking the player to task for being vague.

C. Discussion

The qualitative analysis highlights several key aspects of the method's goals. For instance, the system allows for mid-conversation persona switching, demonstrating the emergent gameplay that Montfort [67] describes as a source of pleasure in interactive fiction. In addition, the game dynamically adjudicates player intent based on situational constraints, effectively balancing player agency with narrative coherence [68]. By imposing structure on otherwise open-ended interactions, the game quantifies player progress toward specified goals, enhancing the overall coherence of the narrative. In the context of recent advancements in LLM-driven gaming [50], [51], [52], our work facilitates measurable goal-oriented outcomes and improvisational capabilities that adapt to player actions.

The quantitative evaluation demonstrates the strengths of our approach, showing that the QALM-based state management improves the coherence and flow of gameplay. By decoupling state management from dialogue generation, our method ensures that player actions are evaluated against clear criteria, leading to more consistent gameplay experiences with minimal out of character interruptions. The puzzle graph framework, in particular, allows for structured progression through the game, maintaining narrative integrity while still accommodating player improvisation. This scaffolding effectively uses the capabilities of the LLM, turning what could be an unstructured interaction into a well-defined and engaging game experience.

The proposed game scenario uses five core stages. The interaction within each state is partially compartmentalized, with additional external guidance provided as required. This has the potential to improve scalability for larger designs. During our evaluation, we employed a context window of 16k tokens. This was sufficient to contain the complete history of focused sessions, while longer sessions exceeded the limit. Experiments with a smaller 4k token context showed that when simply using sliding window truncation, the flow of the game was not affected. This is partially the result of targeted state-level reinforcement and compartmentalized design of each state. However, retaining longer term utterances with important content may require a more advanced memory management protocol, a possible direction for future work. Given the rapid increase in context window sizes, the context window may not represent a long-term challenge. However, the increased computational cost associated with larger context size, along with memory and latency considerations, could be a barrier for implementing long-form interactions using open-source models without state-of-the-art memory management [69], [70].

Rigid and explicit structure limits player choices. In conventional systems, game states are predefined, and dialogue trees prescribe available options. Consequences of choices are predetermined. In contrast, our proposed method encourages diverse player choices, using states as waypoints that guide transitions between emergent states. These emergent states and corresponding dialogue arise dynamically in response to user input during the journey between explicit waypoints. By offering the user the ability to make choices unconstrained by menus, the method supports a high degree of completeness of options. Similarly, through the conditioning of the NPC and the associated continuous tracking and updating of state information, the players' expectations are satisfied. Thus, player agency is improved [7], [9], [10]. Furthermore, previous constraints encountered with prescribed choices are ameliorated, as the LLM performs much of the heavy lifting to enable more freeform interaction.

We term this concept “*hard agency*,” where the player is required to conceptualize and take deliberate action without relying on explicit options and with few affordances. In tandem, the system must support and adapt to the player’s chosen path dynamically, without prior knowledge or predefined outcomes as in our method. This is in contrast to “*soft agency*,” where player may make implicit choices, but the resulting sense of control is largely superficial. For example, in a stealth game, a player may choose between multiple pathways each marked by

affordances such as vents or vulnerable guards. While this setup creates the illusion of choice, the underlying outcomes often converge, limiting the impact of the player’s decisions.

1) *Limitations*: The proof of concept relies on a large-scale commercial LLM application programming interface (API). This is impractical for game studios aiming to publish using the proposed method. However, rapidly advancing open-source models, which can be fine-tuned and run on end-user hardware, offer a viable alternative. Many perform as well as or better than our baseline model across many tasks [71]. Techniques like quantization significantly reduce memory usage and inference times, making these models feasible even for lower end GPUs [72], [73]. However, even with these optimizations, long context windows remain problematic due to the nonlinear scaling of computational requirements. While challenges remain for mobile devices, we do not expect computational requirements to hinder the adoption of LLMs for many game types and devices in the near to medium term.

The DLM does not always adhere to the prompt directives, sometimes causing the NPC to respond out of character. For example, the NPC may attempt to protect the hostages [see Fig. 10(b)] or object in some way to the game situation as if applying a moral judgment, incompatible with the character. Such deviations can be easily corrected at the global or state level, but are only feasibly uncovered over many gameplay testing sessions. Whenever we discovered a deviation, we added a corrective prompt directive to the behavior correction block of the prompt (see Section III-C). With sufficient testing of a game version, most consistency issues can be corrected. However, a persistently adversarial player can eventually cause failures via creative attempts to confuse the DLM through out-of-context utterances or other attacks. Many such attacks can be handled via preprocessing player input, though investigating this fully was out of scope for the proof of concept.

VI. CONCLUSION

In this article, we presented a novel approach to goal-oriented conversational games, integrating structured state management with the flexibility of LLMs. The introduction of the puzzle graph in tandem with the QALM and DLM LLM instances allowed for the measurement of player progress toward objectives, leading to a more coherent and structured interaction.

Our graph-based method structures the game objectives as stages of a puzzle, using closed-ended questions to create a fact-based representation of objective completion. We showed that by evaluating truth values of this representation against dialogue history between a player and adversarial LLM character, we not only accurately track the game state but also make progress on the challenges of player agency and narrative coherence that are often encountered in open-ended interactions.

To evaluate our method, we created a new type of game that extends the concept of the conversational puzzle. We demonstrated that an appropriately conditioned LLM-based character can serve as a realistic adversary, providing plausible dialogue while also adapting and reacting to diverse and creative player improvisation. This is a significant improvement when compared

to stand-alone LLMs that often produce shapeless meandering interactions.

We supported these claims with a detailed qualitative analysis of gameplay interactions and a quantitative evaluation of the state tracking performance of our method compared to baseline LLM-generated state predictions.

Building on the findings of this work, several potential directions are possible. One critical area involves expanding the application of the puzzle graph and QALM framework to more diverse and complex game scenarios. Using the method to manage interactions across multiple characters while incorporating dynamic changes to objectives based on events in a wider game context is a natural next step. Similarly, embedding LLM-enabled interactions within more conventional games (especially, but not limited to, CRPGs) would create more lively and believable settings and gameplay.

In addition, improving the strategic reasoning capabilities of the DLM is an interesting avenue that could pose more elaborate challenges to players. This could involve the development of more sophisticated training methodologies to enable more nuanced and contextually appropriate decision making in adversarial environments. Finally, addressing the ethical implications and safety concerns associated with AI-driven gameplay, particularly in maintaining alignment and avoiding unintended outcomes, represents an important consideration when developing such characters.

ACKNOWLEDGMENT

The authors would like to thank the reviewers for their thoughtful comments and suggestions.

REFERENCES

- [1] K. Salen Tekinbaş and E. Zimmerman, "Rules of play," in *Game Design Fundamentals*. Cambridge, MA, USA: MIT Press, 2004.
- [2] V.-M. Karhulahti, "Fiction puzzle: Storable challenge in pragmatist videogame aesthetics," *Philos. Technol.*, vol. 27, no. 2, pp. 201–220, Jun. 2014.
- [3] J. Lessard, *A Tough Sell*. San Francisco, CA, USA: LabLabLab, 2014.
- [4] J. Lessard, "Designing natural-language game conversations," in *Proc. Int. Joint Conf. DiGRA-FDG*, 2016, pp. 3–4.
- [5] Latitude, "AI Dungeon," 2019. [Online]. Available: <https://play.aidungeon.io>
- [6] N. Weir, R. Thomas, R. D'Amore, K. Hill, B. Van Durme, and H. Jhamtani, "Ontologically faithful generation of non-player character dialogues," *EMNLP*, pp. 9212–9242, 2024.
- [7] J. Murray, *Hamlet on the Holodeck: The Future of Narrative in Cyberspace*. Cambridge, MA, USA: MIT Press, 2017.
- [8] G. Calleja, *In-Game: From Immersion to Incorporation*, 1st ed. Cambridge, MA, USA: MIT Press, 2011.
- [9] N. Wardrip-Fruin, M. Mateas, S. Dow, and S. Sali, "Agency reconsidered," in *Proc. DiGRA Int. Conf. Digit. Games Res. Assoc.*, 2009.
- [10] K. Tanenbaum and T. J. Tanenbaum, "Agency as commitment to meaning: Communicative competence in games," *Digit. Creativity*, vol. 21, no. 1, pp. 11–17, 2010.
- [11] W. Crowther and D. Woods, "Colossal Cave Adventure," *Video Game*, 1976.
- [12] Infocom, *Zork*, Video Game, 1980.
- [13] W. A. Woods, "Transition network grammars for natural language analysis," *Commun. ACM*, vol. 13, pp. 591–606, 1970.
- [14] A. V. Aho and J. D. Ullman, *The Theory of Parsing, Translation, and Compiling. 1: Parsing*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1972.
- [15] A. Plotkin, "Rule-based programming in interactive fiction," Web Page, 2023.
- [16] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, "Search-based procedural content generation: A taxonomy and survey," *IEEE Trans. Comput. Intell. AI Games*, vol. 3, no. 3, pp. 172–186, Sep. 2011.
- [17] G. N. Yannakakis and J. Togelius, "Experience-driven procedural content generation," *IEEE Trans. Affect. Comput.*, vol. 2, no. 3, pp. 147–161, Jul./Sep. 2011.
- [18] S. Värtinen, P. Hämäläinen, and C. Guckelsberger, "Generating role-playing game quests with GPT language models," *IEEE Trans. Games*, vol. 16, no. 1, pp. 127–139, Mar. 2024.
- [19] M. Cavazza, F. Charles, and S. J. Mead, "Character-based interactive storytelling," *IEEE Intell. Syst.*, vol. 17, no. 4, pp. 17–24, Jul./Aug. 2002.
- [20] D. Thue, V. Bulitko, M. Spetch, and E. Wasylishen, "Interactive storytelling: A player modelling approach," in *Proc. 3rd Artif. Intell. Interact. Digit. Entertain. Conf.*, 2007, pp. 43–48.
- [21] G. N. Yannakakis, P. Spronck, D. Loiacono, and E. Andre, "Player modeling," in *Proc. Artif. Comput. Intell. Games*, 2013, pp. 45–59.
- [22] C. Klimmt, T. Hartmann, and A. Frey, "Effectance and control as determinants of video game enjoyment," *Cyberpsychol. Behav. Soc. Netw.*, vol. 10, no. 6, pp. 845–848, 2007.
- [23] Bethesda Game Studios, "Starfield," Video Game, 2023.
- [24] D. Cage, "Fahrenheit: Indigo Prophecy," Video Game, 2015.
- [25] Supermassive Games, "Until dawn," Video Game, 2015.
- [26] Dontnod Entertainment, "Life is strange," Video Game, 2015.
- [27] K. Riach and S. Stuard, "Oxenfree," Video Game, 2016.
- [28] M. Mateas and A. Stern, "Facade," Video Game, 2005.
- [29] M. Mateas and A. Stern, "Facade: An experiment in building a fully-realized interactive drama," in *Proc. Game Developers Conf.*, 2003.
- [30] Ocelot Society, "Event[0]," Video Game, 2016.
- [31] E. Short, "Blood & Laurels," Video Game, 2014.
- [32] J. Lessard, "Design rationale for natural language game conversations," in *Proc. 10th Int. Conf. Found. Digit. Games*, Pacific Grove, CA, USA, 2015.
- [33] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, Lake Tahoe, NV, USA, 2013, pp. 3111–3119.
- [34] J. Pennington, R. Socher, and C. Manning, "Glove: Global vectors for word representation," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2014, pp. 1532–1543.
- [35] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [36] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, Montreal, QC, Canada, 2014, pp. 3104–3112.
- [37] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Ch. Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2019, pp. 4171–4186.
- [38] A. Vaswani et al., "Attention is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, Long Beach, CA, USA, 2017, pp. 5998–6008.
- [39] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, 2019.
- [40] I. Millington and J. Funge, *Artificial Intelligence for Games*, 2nd ed. San Mateo, CA, USA: Morgan Kaufmann, 2009.
- [41] P. García-Sánchez, G. N. Yannakakis, and J. Togelius, *Artificial Intelligence and Games*. Springer, 2018, print ISBN: 978-3-319-63518-7, online ISBN: 978-3-319-63519-4, <https://doi.org/10.1007/978-3-319-63519-4>, Genet. Program. Evolvable Mach., vol. 20, no. 1, pp. 143–145, 2019.
- [42] Y. A. Sekhavat, "Behavior trees for computer games," *Int. J. Artif. Intell. Tools*, vol. 26, no. 2, 2017, Art. no. 1730001.
- [43] M. O. Riedl and R. M. Young, "Narrative planning: Balancing plot and character," *J. Artif. Intell. Res.*, vol. 39, pp. 217–268, 2010.
- [44] S. G. Ware, E. T. Garcia, M. Fisher, A. Shirvani, and R. Farrell, "Multiagent narrative experience management as story graph pruning," *IEEE Trans. Games*, vol. 15, no. 3, pp. 378–387, Sep. 2023.
- [45] S. Ji, S. Pan, E. Cambria, P. Martinen, and P. S. Yu, "A survey on knowledge graphs: Representation, acquisition, and applications," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 2, pp. 494–514, Feb. 2022.
- [46] T. Ashby, B. K. Webb, G. Knapp, J. Searle, and N. Fulda, "Personalized quest and dialogue generation in role-playing games: A knowledge graph-and language model-based approach," in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, 2023, Art. no. 290.

- [47] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.
- [48] A. Sarkar and S. Cooper, "Procedural content generation using behavior trees (PCGBT)," in *Proc. Joint AIIDE Workshops*, 2021.
- [49] D. Ghosh, A. Gupta, and S. Levine, "Learning actionable representations with goal conditioned policies," in *Proc. 7th Int. Conf. Learn. Represent.*, New Orleans, LA, USA, 2019, pp. 2–5.
- [50] Y. Liu, P. Sun, and H. Li, "Large language models as agents in two-player games," 2024, *arXiv:2402.08078*.
- [51] S. R. Cox and W. T. Ooi, "Conversational interactions with NPCs in LLM-driven gaming: Guidelines from a content analysis of player feedback," in *Chatbot Research and Design, Conversations*. Berlin, Germany: Springer, 2023, pp. 167–184.
- [52] R. Gallotta et al., "Large language models and games: A survey and roadmap," *IEEE Trans. Games*, doi: [10.1109/TG.2024.3461510](https://doi.org/10.1109/TG.2024.3461510).
- [53] C. Cheng et al., "In dialogues we learn": Towards personalized dialogue without pre-defined profiles through in-dialogue learning," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Miami, FL, USA, 2024, pp. 10408–10422. [Online]. Available: <https://aclanthology.org/2024.emnlpmain.581>
- [54] J. Freiknecht and W. Effelsberg, "Procedural generation of interactive stories using language models," in *Proc. Int. Conf. Found. Digit. Games*, 2020, pp. 97:1–97:8.
- [55] Q. Wang et al., "Recursively summarizing enables long-term dialogue memory in large language models," 2023, *arXiv:2308.15022*.
- [56] Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, "Calibrate before use: Improving few-shot performance of language models," in *Proc. 38th Int. Conf. Mach. Learn.*, 2021, pp. 12697–12706.
- [57] Q. Chen, Z. Zhuo, and W. Wang, "BERT for joint intent classification and slot filling," 2019, *arXiv:1902.10909*.
- [58] J. D. Williams, A. Raux, and M. Henderson, "The dialog state tracking challenge series: A review," *Dialogue Discourse*, vol. 7, no. 3, pp. 4–33, 2016.
- [59] N. Mrksic et al., "Multi-domain dialog state tracking using recurrent neural networks," in *Proc. 53rd Annu. Meeting Assoc. Comput. Linguistics*, 2015, pp. 794–799.
- [60] L. Liao, L. H. Long, Y. Ma, W. Lei, and T.-S. Chua, "Dialogue state tracking with incremental reasoning," *Trans. Assoc. Comput. Linguistics*, vol. 9, pp. 557–569, 2021.
- [61] R. Plutchik, "A psychoevolutionary theory of emotion," *Soc. Sci. Inf.*, vol. 21, nos. 4/5, pp. 529–553, 1982.
- [62] Y. Huang et al., "Advancing transformer architecture in long-context large language models: A comprehensive survey," 2023, *arXiv:2311.12351*.
- [63] M. J. McMains and F. J. Lanceley, "The use of crisis intervention principles by police negotiators," *J. Police Crisis Negotiations*, vol. 3, no. 1, pp. 3–30, 2003.
- [64] T. B. Brown et al., "Language models are few-shot learners," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.
- [65] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
- [66] W.-L. Chiang et al., "Vicuna: An open-source chatbot impressing GPT-4 with 90% ChatGPT quality," Mar. 2023.
- [67] N. Montfort, *Twisty Little Passages: An Approach to Interactive Fiction*. Cambridge, MA, USA: MIT Press, 2005.
- [68] E. Carstendottir et al., "Naked and on fire": Examining player agency experiences in narrative-focused gameplay," in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, New York, NY, USA, 2021, pp. 1–13.
- [69] W. Kwon et al., "Efficient memory management for large language model serving with pagedattention," in *Proc. 29th Symp. Oper. Syst. Princ.*, 2023, pp. 611–626.
- [70] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *J. Mach. Learn. Res.*, vol. 23, 2022, Art. no. 120.
- [71] J. Zhao et al., "LoRA Land: 310 fine-tuned LLMs that rival GPT-4, a technical report," 2024, *arXiv:2405.00732*.
- [72] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Proc. Adv. Neural Inf. Process. Syst.*, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds. 2022. [Online]. Available: <https://openreview.net/forum?id=dXiGWqBoxaD>
- [73] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," 2022, *arXiv:2210.17323*.